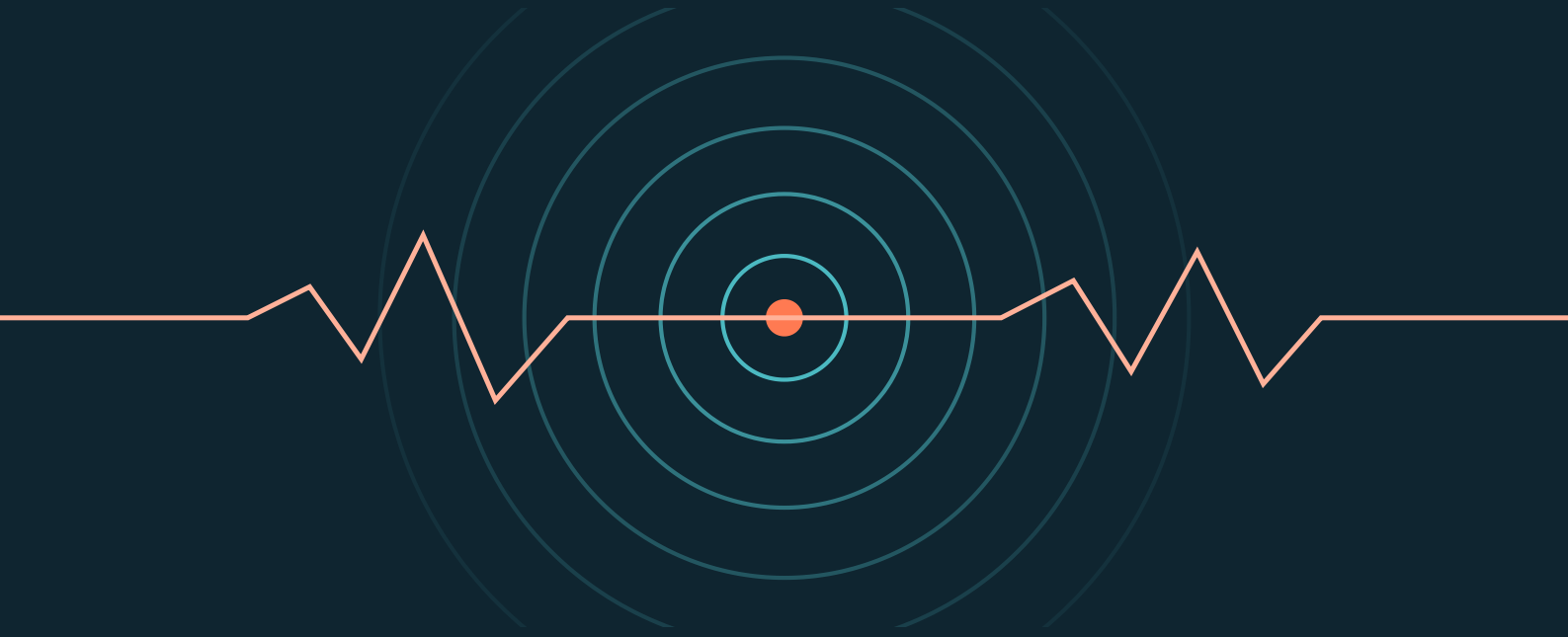
PROJECT REPORT

Disaster Connect

A local-first, offline-first alert system for floods, earthquakes, storms and tsunamis

The full workflow, every tool we used, and the reason behind each choice, from A to Z, explained in simple words.



Python · FastAPI · LangGraph · SQLite · Vanilla JS

September 2026

Prepared by: _____

CONTENTS

Table of contents

	Abstract and how to read	3
1	Introduction	4
2	Background: the four dangers	6
3	Requirements and design principles	8
4	System architecture	10
5	The workflow, step by step	12
6	Sensors and simulation	15
7	The four detectors	17
8	Working with and without the internet	20
9	Alerts and storage	22
10	The backend server	24
11	The user app	26
12	The A–Z register: everything we used, and why	28
13	Packaging: from a folder to “just double-click”	32
14	Testing and results	34
15	Security, privacy and honest limits	36
16	Future work and conclusion	37
	Glossary and references	38

Every chapter starts on a new page. The A–Z chapter is a quick reference to every tool, library and idea used in the project.

SUMMARY

Abstract

A small program that watches the world for trouble, decides in a fraction of a second whether it is dangerous, and rings the alarm, even when the internet is gone.

Floods, earthquakes, storms and tsunamis rarely give a second chance. When they strike, mobile networks and internet links are often the first things to fail. **Disaster Connect** (the code name is *Local Disaster Watch*) is a software system built around that fact. It reads six kinds of sensor data every five seconds, compares them with safety limits for four hazards, and raises an alert when a limit is crossed. The decision is made by short, readable rules, not by a black box, so every alert can say *why* it was raised.

The internet is treated as a bonus, never a need. When a connection exists, the system adds live weather from Open-Meteo, nearby shelters from OpenStreetMap, a phone push message, and a one-sentence explanation written by a language model on the Groq service. When it does not, the buzzer and SMS path still works, and every reading is saved on the device and marked “not synced” until the connection returns.

The software is written in Python. A FastAPI server exposes the decision pipeline as a small web API and serves a phone-style web app with five screens (Home, Shelter, SOS, Prediction, Report). Readings and alerts live in a single SQLite file. The whole product is packaged as one Windows zip file that carries its own copy of Python, so a person can unzip it, double-click `START.bat`, and have the app open in their browser.

We tested the product by loading all 14 code modules, running a full detection cycle offline, calling every web address, feeding the storm and tsunami detectors known dangerous values, and running the zip from a clean folder. Every check passed. The report also lists honest limits: the sensors are simulated, SMS messages are only printed to a log, and this is a learning project, not an official warning service.

How to read this report

Each chapter answers one question:

- **Chapters 1–4**: what problem, what goal, what design.
- **Chapters 5–9**: how it works, step by step.
- **Chapters 10–11**: the server and the screens.
- **Chapter 12**: the A–Z list of everything used and why.
- **Chapters 13–14**: packaging and testing.
- **Chapters 15–16**: limits, and what comes next.

> **IN SIMPLE WORDS**
> Look for the blue “In simple words” boxes for a plain explanation, and the red “Why we chose this” boxes for the reason behind a decision.

Keywords: disaster alert, offline-first, rule-based detection, STA/LTA, FastAPI, LangGraph, SQLite, Open-Meteo, portable application.

CHAPTER 1

Introduction

1.1 The problem

Think of a smoke alarm. It is cheap, it sits quietly on a ceiling, and it does one job: it screams when there is smoke. It does not need Wi-Fi, an app or a phone call to do that, which is exactly why people trust it.

Natural dangers are harder. A flood, an earthquake, a storm and a tsunami each show up in a different way, and each needs different measuring. Most warning apps depend on a server far away and a phone signal near you. In a real emergency, that is the weakest link. Towers lose power, networks jam, and an app that only works online goes silent at the worst time.

> **IN SIMPLE WORDS**
> We wanted a smoke alarm for many dangers at once: it watches the numbers, decides by itself, and only uses the internet for extras.

1.2 Objectives

1. Read sensors on a fixed schedule (every 5 seconds) and store each reading.
2. Detect four hazards (flood, earthquake, storm, tsunami) with rules that a person can read and check.
3. Raise an alert through a buzzer and SMS path that *never* needs the internet.
4. Use the internet, when present, to add weather, shelters, push messages and a friendly sentence.
5. Show everything in a simple phone-style app, including nearby safe places and an SOS screen.
6. Ship as a single zip that anyone can start without installing anything.

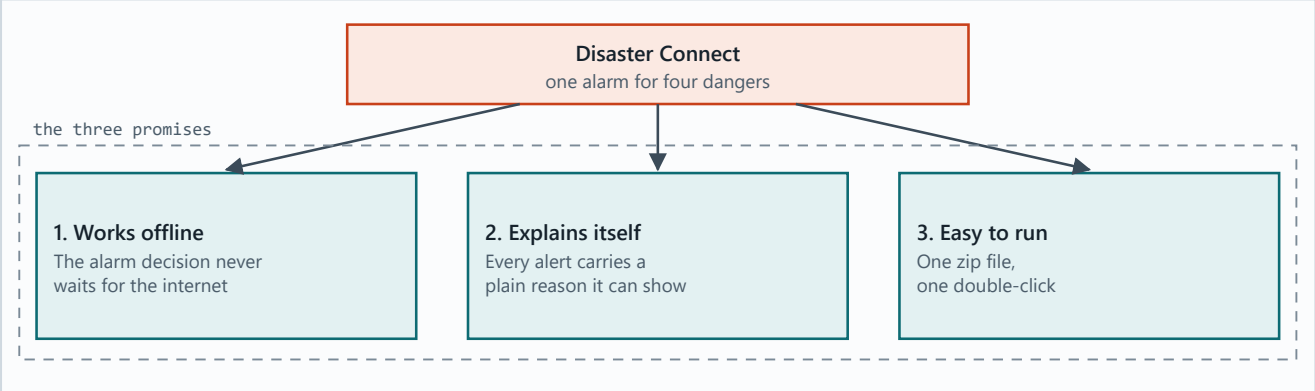


Figure 1. The three promises that shaped every design choice in the project.

1.3 Scope

Inside the project

- Four hazard detectors with configurable limits
- A simulated sensor layer, with real-hardware code ready to switch on
- Offline storage and “sync on reconnect”
- A FastAPI backend and a mobile-style web app
- Optional weather, shelter and language-model extras
- A portable Windows package and automatic tests in CI

Outside the project

- Predicting when a disaster will happen
- Replacing official warning agencies
- Real SMS delivery (a hook exists, the gateway does not)
- Background phone notifications when the app is closed
- Sensor calibration for a real site

1.4 How this compares with a smoke alarm

Table 1. The smoke-alarm idea, extended to four hazards.

IDEA	SMOKE ALARM	DISASTER CONNECT
What it senses	Smoke particles	Water level, rain, ground shaking, wind, sea level, pressure
How it decides	One fixed threshold	Four rule sets, worst result wins
Needs internet?	No	No for the alarm, yes for the extras only
How it warns	Loud sound	Buzzer, SMS, push message, on-screen risk
Explains itself?	No	Yes, every alert states its reason
Keeps a history?	No	Yes, in a local database

1.5 Who this is for

The report is written for a mixed reader: a teacher marking the project, a classmate who wants to run it, and anyone who wants to understand the choices behind it. Technical words are always explained the first time they appear, and the glossary on the second-last page collects them all.

1.6 Project at a glance

Table 2. Key facts about the finished project.

Language	Python 3.13 (server) and plain JavaScript (screens)
Hazards	Flood, earthquake, storm and thunderstorm, tsunami
Check interval	5 seconds (`SENSOR_POLL_SECONDS`)
Storage	One SQLite file with two tables
Server	FastAPI, port 5000
Code modules	14 modules that load without error
Delivery	28 MB Windows zip with Python bundled

CHAPTER 2

Background: the four dangers

Before writing code we asked a simple question for each danger: *what can a machine measure that tells us it is starting?* The answers decide which sensors the project needs.

2.1 Flood

A flood happens when too much water arrives too fast, either from heavy rain or from a river rising. Two numbers matter most: how high the water is, and how hard it is raining. Our water sensor is an ultrasonic sensor that points down at the water. It measures the **distance from the sensor to the water surface**, so a *smaller* number means the water is *higher* and closer to danger.

2.2 Earthquake

An earthquake is the ground shaking. The ground is never perfectly still (trucks pass, people walk), so a fixed number is a poor test. Seismologists compare the shaking *right now* with the shaking *usually*. A sudden jump is the signal. An accelerometer measures the shaking as “g”, the same unit as gravity.

2.3 Storm and thunderstorm

Storms are dangerous mainly because of strong wind and extreme rain. A wind sensor and a rain gauge measure both. A weather service can also report a thunderstorm directly through a standard weather code.

2.4 Tsunami

A tsunami is a giant wave set off by an earthquake under the sea. On the coast it shows up as a rapid rise or fall of the sea level. Only a coastal **tide gauge** can see it. A river or pond sensor is the wrong instrument, which is why this detector is built to be strict.

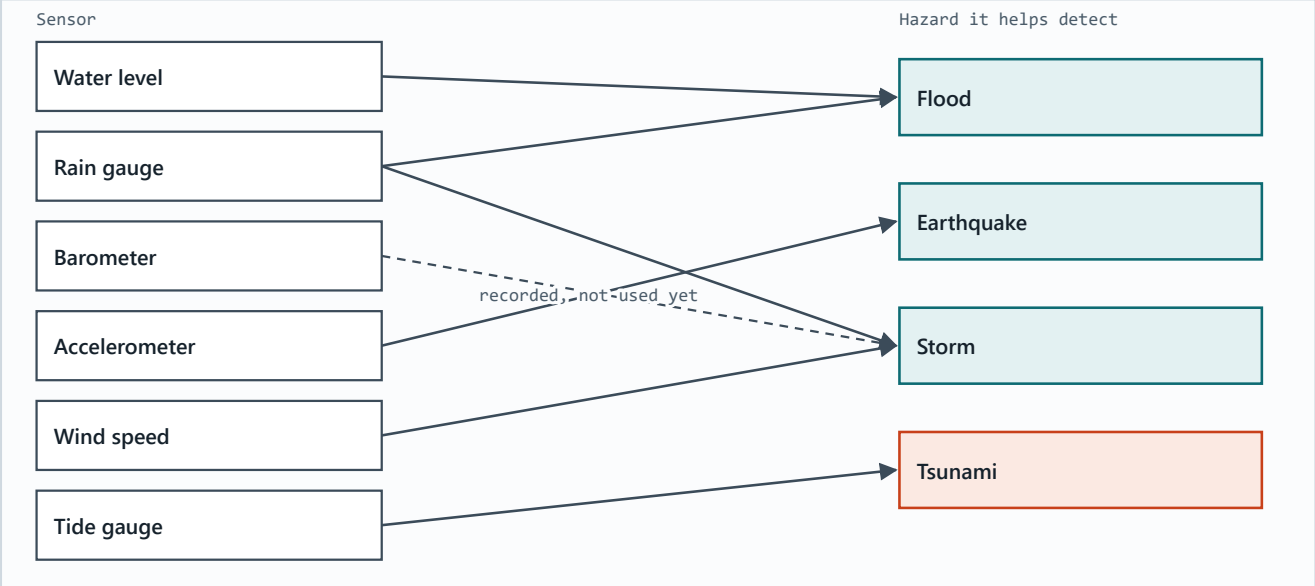


Figure 2. Which sensor feeds which hazard. Air pressure is recorded and shown, but no detector uses it yet.

2.5 Sensors, units and what “normal” looks like

Table 3. The six readings taken every cycle, with real sensor parts named for a future hardware build.

READING	UNIT	REAL PART (PLANNED)	QUIET VALUE	DANGER VALUE
Water level	cm to water	HC-SR04 ultrasonic	`90–150`	`20–60`
Rain rate	mm/hour	Tipping-bucket gauge	`0–15`	`35–70`
Air pressure	hPa	BME280	`1008–1015`	`995–1002`
Vibration	g	MPU6050	`0–0.03`	`0.4–1.2`
Wind speed	km/h	Anemometer	`3–28`	`75–115`
Sea-level change	cm	Coastal tide gauge	`−3 to +3`	`55–120`
Location	lat / lon	NEO-6M GPS	around Pune, India (simulated)	

> **IN SIMPLE WORDS**
> A sensor is like a thermometer for something other than heat. Each one turns a real-world thing (water height, wind, shaking) into a number the computer can read.

2.6 Why these four hazards together

These four are linked. A storm brings the rain that causes a flood. An undersea earthquake causes a tsunami. Watching them in one program means one set of settings, one database, one alert path, and one screen for the user, instead of four separate apps.

2.7 What other systems do

Table 4. Common approaches and where Disaster Connect differs.

APPROACH	STRENGTH	WEAKNESS	OUR ANSWER
Weather app	Wide coverage	Needs internet; no local sensors	Use it only as an extra
Government alerts	Official, trusted	Needs a phone signal; not local	Complement, not replace
Cloud AI service	Powerful	Slow, costs money, needs internet	Rules decide; AI only reworded text
Single-sensor alarm	Simple, offline	One hazard only	Four hazards, one program

> **WHY WE CHOSE THIS**
> **Local first.** A measurement taken next to you, a few seconds ago, is more valuable in an emergency than a perfect forecast that needs a network to arrive.

CHAPTER 3

Requirements and design principles

3.1 What the system must do

Table 5. Requirements and how the project meets them.

#	REQUIREMENT	HOW IT IS MET
R1	Decide with no internet	All four detectors are local rule checks
R2	Say why an alert was raised	Each detector returns a `reason` string
R3	Never crash on a network error	Every online call returns `None` or `[]` on failure
R4	Remember what happened	SQLite tables `readings` and `alerts`
R5	Catch up after an outage	A `synced` flag and a sync step on reconnect
R6	Tuned without editing logic	All limits sit in `config/settings.py`
R7	Usable by non-programmers	Phone-style web app and a one-click zip

3.2 Five design principles

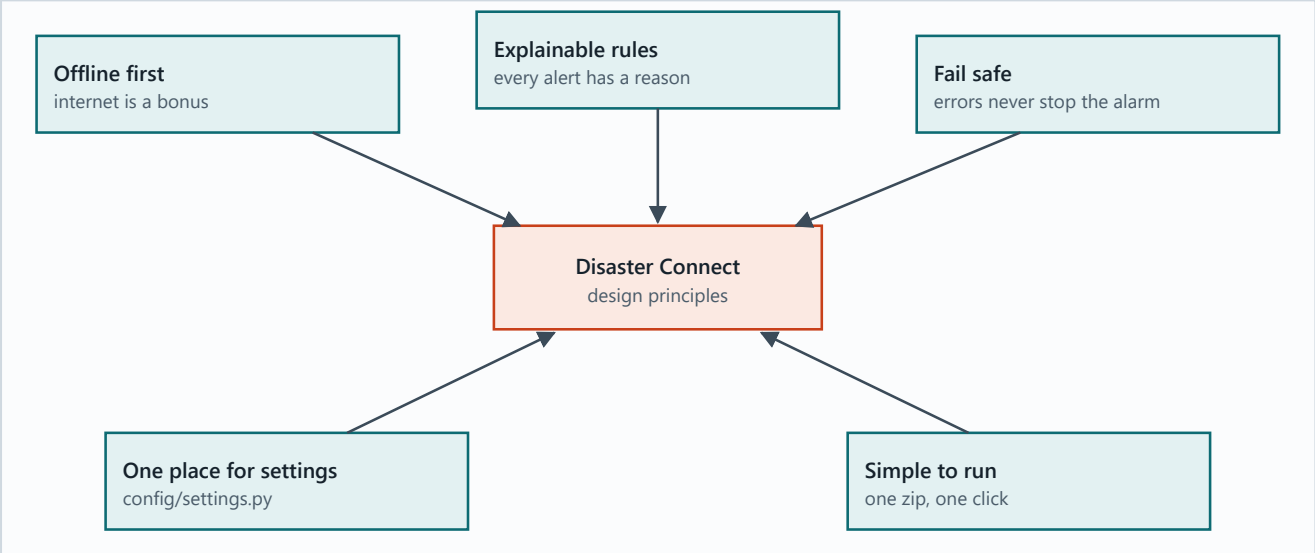


Figure 3. Five principles that were applied to every module. When two ideas conflicted, the earlier one won.

Offline first. Anything that can fail without a network must not be on the critical path.

Explainable rules. A person should be able to read the detector, find the limit, and agree or disagree with it.

Fail safe. A missing internet link, a slow server, or a broken language-model call must lower the *extras*, never the alarm. Each network function catches its own errors and returns a harmless empty value.

One place for settings. Limits, paths, timing and switches live in a single file, so tuning for a new location needs no code changes.

Simple to run. The most correct program is useless if nobody can start it. This principle drove the portable zip.

3.3 Rules, machine learning or a language model?

A common question: “why not use AI to detect disasters?”. We compared three ways of making the decision.

Table 6. Three ways to decide, and why rules won for the alarm itself.

	RULES	MACHINE-LEARNING MODEL	LANGUAGE MODEL
Speed	Instant	Fast after training	Seconds, over the network
Works offline	Yes	Yes, if bundled	No
Explainable	Fully	Poorly	Unreliably
Needs training data	No	Lots of real events	No
Can invent facts	No	Can be wrong quietly	Yes
Cost	Free	Data + tuning effort	Per-call cost, key needed

WHY WE CHOSE THIS

Rules decide, the language model only rewords. A wrong “low” from a model could cost lives, and we have no real disaster data to train one. So the decision is pure rules. The language model may make the sentence friendlier, and the code is written so that it can never change the risk level.

IN SIMPLE WORDS

Rules are like a recipe: do exactly these steps, get the same cake every time. A language model is like a chef improvising, creative but not something you want deciding whether a fire alarm should ring.

3.4 Trade-offs we accepted

- **Fixed limits can be wrong for a place.** A limit right for Pune may be wrong elsewhere. Settings are easy to change to compensate.
- **Simulated sensors.** Development speed was more important than hardware. The real sensor code is written as comments and ready to switch on.
- **A framework for a short chain.** LangGraph adds a dependency for what is, today, a straight line of steps. It keeps the door open for branching later.

CHAPTER 4

System architecture

The system is built as four layers. Information flows from the top layer down. Each layer only talks to the one beside it, so any part can be swapped without breaking the others, for example simulated sensors for real ones.

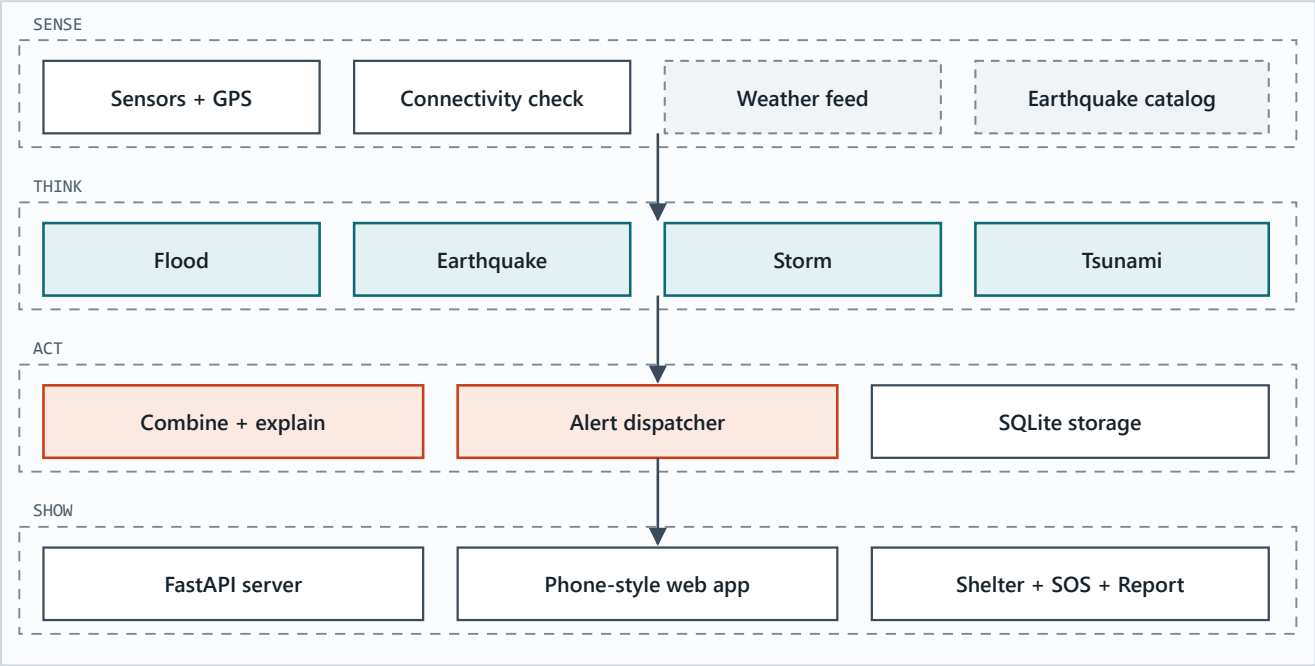


Figure 4. Four layers. Dashed boxes are optional online extras; solid boxes work with no internet.

Table 7. What each layer does.

LAYER	JOB	CODE THAT DOES IT
Sense	Collect readings and context	`sensors/ network/`
Think	Turn numbers into “low / high”	`agents/`
Act	Warn people, remember what happened	`alerts/ database/`
Show	Display it to a person	`backend/ frontend/`

> **IN SIMPLE WORDS**
> Sense = eyes and ears. Think = the brain. Act = the hands and the bell. Show = the face people look at.

4.1 The folder map

The project follows the rule “each folder does one job”. Opening `agents/` shows only decision logic; opening `network/` shows only things that talk to the internet.

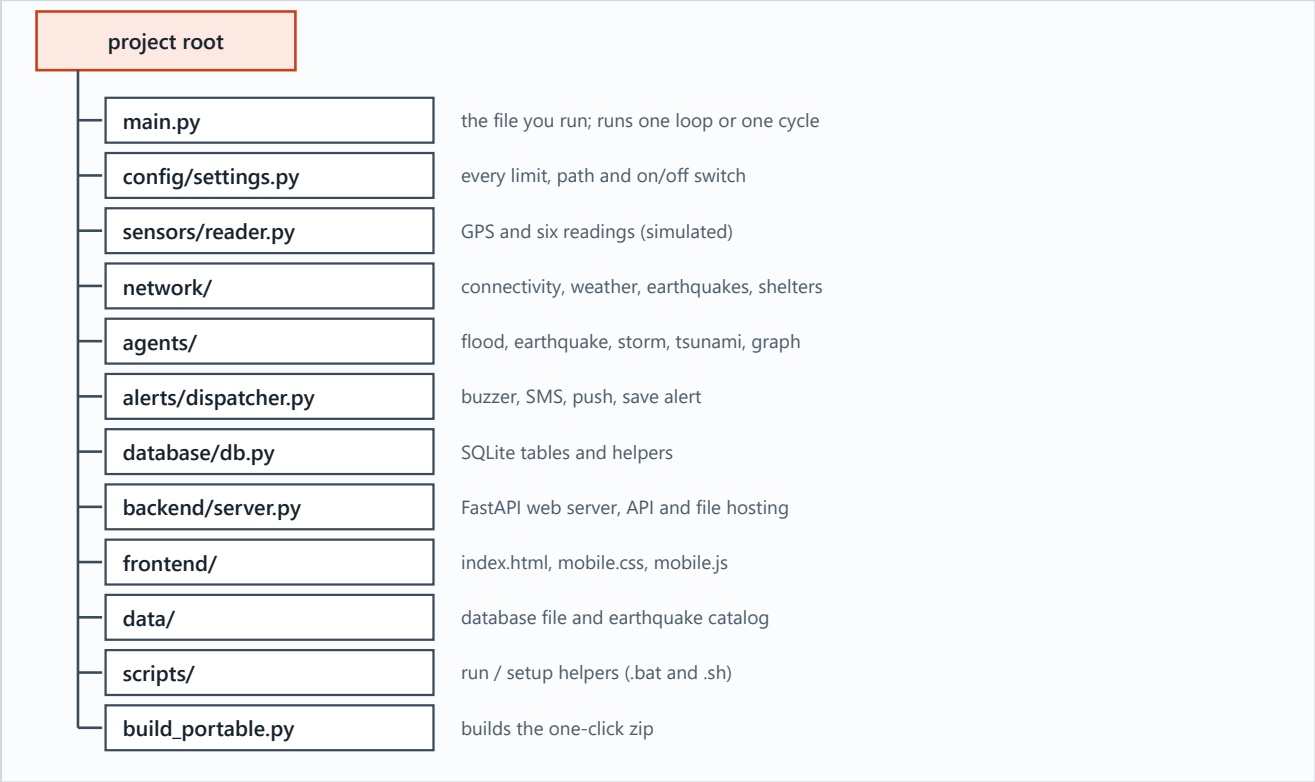


Figure 5. Where each responsibility lives in the project folder.

> WHY WE CHOSE THIS
>
> **Small, single-purpose folders.** When the storm and tsunami detectors were added, only `agents/`, one line in `config/` and the reading in `sensors/` changed. Nothing else needed touching.

4.2 Adding a new hazard: the recipe

Because each job has its own home, adding a hazard follows the same short recipe. This is how storm and tsunami were added.

1. Add its limit to `config/settings.py`.
2. Add its reading to `sensors/reader.py` and its column to `database/db.py` (the database upgrades itself).
3. Write a small file in `agents/` that returns a risk and a reason.
4. Add one node to the chain in `agents/graph.py` and place it in the priority order.
5. Show the new answer on the Home screen.

CHAPTER 5

The workflow, step by step

This chapter follows one cycle of the system from the moment it wakes up to the moment it goes back to sleep.

5.1 The main loop

The heart of the program is the function `run_loop()` in `main.py`. It repeats the twelve steps below forever, once every five seconds. Run with the `--once` flag it does a single round and stops, which is what the automatic tests use.

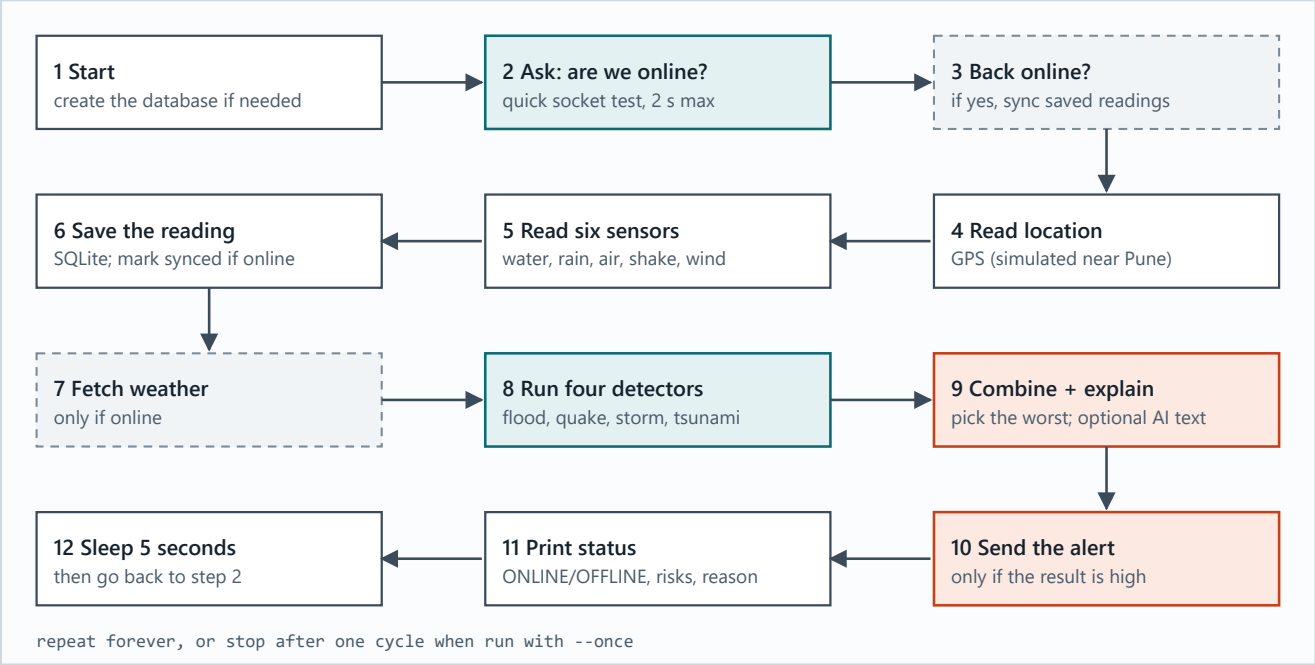


Figure 6. The main loop drawn as a snake: read left to right on rows 1 and 3, right to left on rows 2 and 4.

IN SIMPLE WORDS

Every 5 seconds the program looks around, writes down what it sees, thinks about whether it is dangerous, and rings a bell if it is. Then it takes a short nap.

WHY WE CHOSE THIS

A simple loop, not many threads. One loop is easy to read, easy to test, and cannot fight with itself over the database. Five seconds is fast enough for slow hazards like floods, and cheap enough to run on a tiny computer.

5.2 One request through the web server

When the phone-style app opens, it sends its position to the server and asks for a risk answer. Figure 7 shows every call the server makes to answer that one request. The same sequence runs inside `main.py`, only without the phone and the JSON reply.

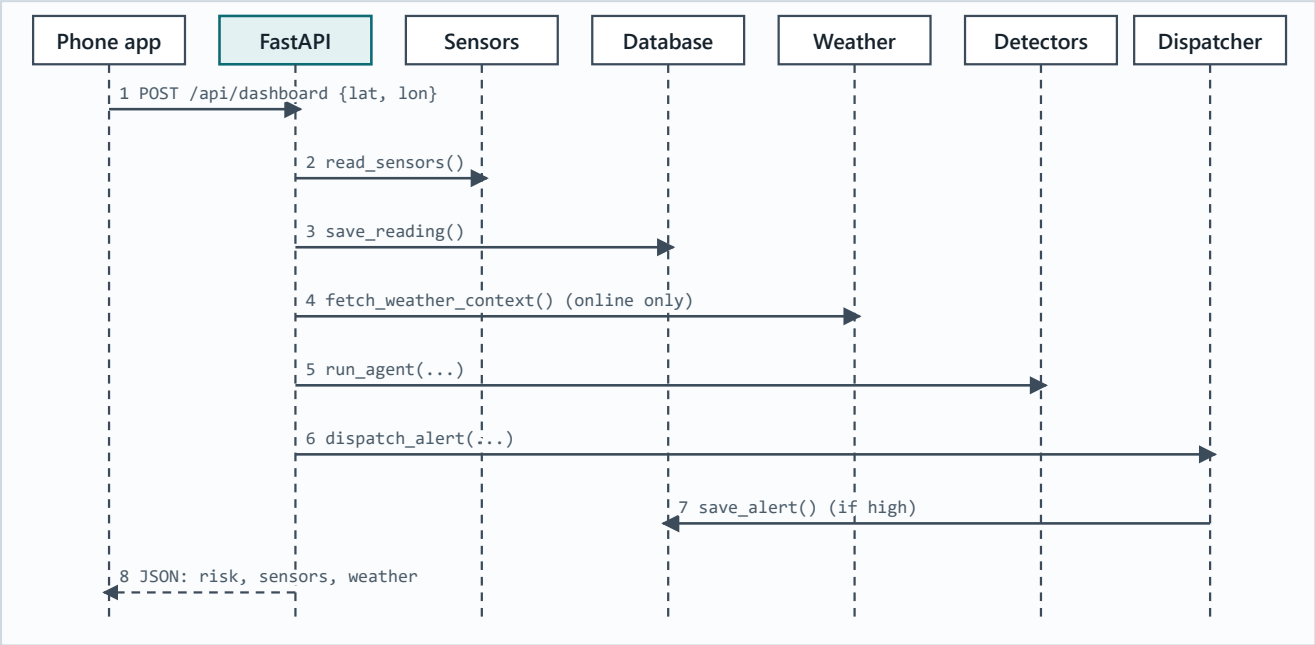


Figure 7. Sequence of one `POST /api/dashboard` call. Time runs downwards. The dashed reply goes back to the phone.

Table 8. What each step protects against.

STEP	WHAT COULD GO WRONG	WHAT THE CODE DOES
3	Database file missing	`init_db()` creates it at server start
4	No internet, slow API	3 s timeout, returns `None`
5	Weather is `None`	Detectors handle a missing weather value
6	Result is low	Dispatcher returns at once, nothing sent

5.4 How long can one cycle take?

Every online step has a time limit, so a slow network can never freeze the loop for long. The table adds up the worst case when everything is slow at the same time.

Table 9. Time limits inside one cycle.

STEP	LIMIT	APPLIES WHEN
Connection test	2 s	every cycle in auto mode
Weather request	3 s	online only
Language-model sentence	4 s	online, key set, alert is high
Sleep between cycles	5 s	always

> **IN SIMPLE WORDS**
> Worst case, one round takes about 14 seconds (2 + 3 + 4 + 5). With no internet it is just the 5-second nap, because the online steps are skipped.

5.3 The shared state: one dictionary that grows

The detectors run as a chain in **LangGraph**. The chain passes along one shared dictionary called the *state*. It starts with the inputs. Each step reads what it needs and adds its own answer. By the end, the dictionary holds everything: each detector's verdict, the overall risk, the hazard name and the reason.

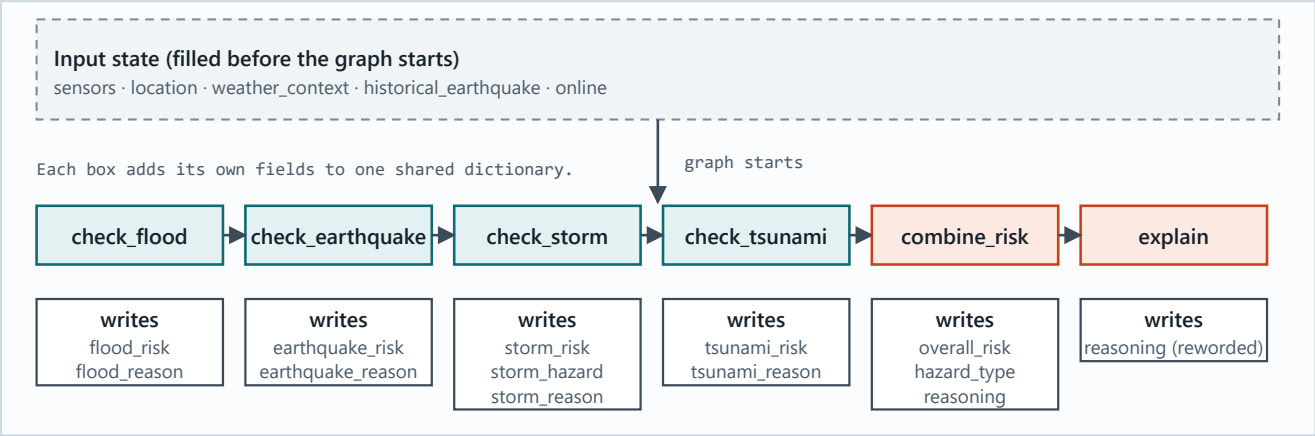


Figure 8. Six nodes in order. The first four add a risk and a reason each; `combine_risk` picks the winner; `explain` may reword the sentence.

> **IN SIMPLE WORDS**
> Imagine a form passed down a line of people. Each person fills in their own box and hands it on. At the end, the form is complete.

Two ways to run the chain

- `run_agent`: the live path. The earthquake check remembers the last 200 vibration readings from earlier cycles.
- `run_agent_isolated`: the test path. It scores vibration against a fixed quiet baseline, so the same input always gives the same answer, no matter what ran before.

> **WHY WE CHOSE THIS**
> **Why a graph library at all?** Today the chain is a straight line, so plain function calls would also work. LangGraph was chosen so branches can be added later (for example, only call the tsunami check at coastal sites) without rewriting the flow. The cost is one extra dependency.

> **Safety net.** Rules always run first. The last node, `explain`, is wrapped in a broad try/except. If the language model fails for any reason, the rule sentence stays.

CHAPTER 6

Sensors and simulation

Every cycle begins by reading the world. Because a real sensor kit was not available for development, `sensors/reader.py` **simulates** them. The simulator is not a random number generator that ignores physics. It is built so that most of the time everything is calm, and now and then a believable danger appears.

6.1 How the simulator creates danger

Each cycle the simulator rolls one random number between 0 and 1, called `danger_roll`. Different parts of that number line trigger different dangers, as Figure 9 shows. The parts overlap on purpose, because real disasters can happen together (a storm brings both wind and rain).

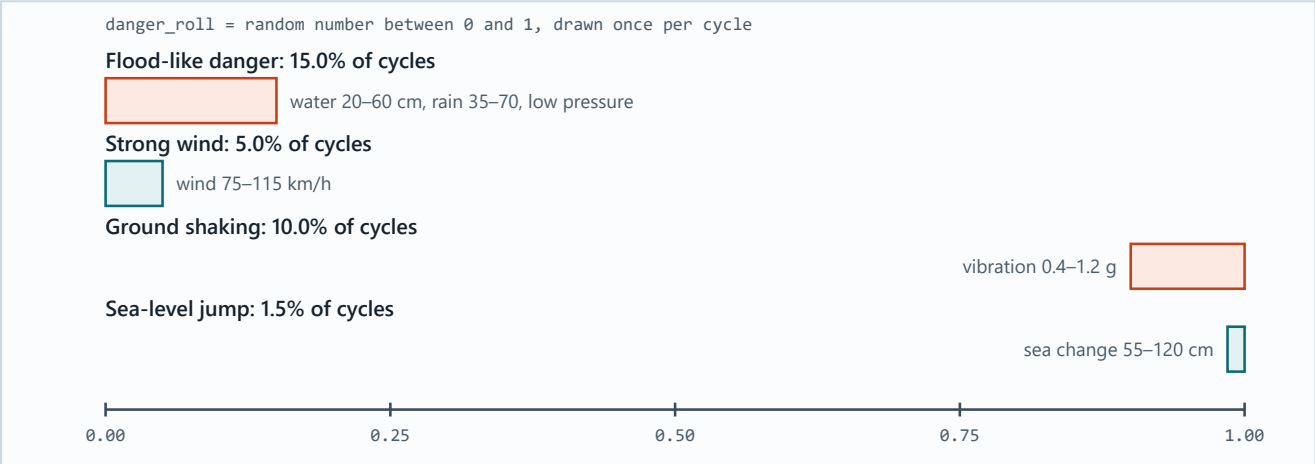


Figure 9. The simulator's danger map. Percentages come straight from the code's thresholds (for example, a roll below 0.15 creates flood-like values).

What this means for a demo

About one cycle in seven looks like a flood, one in ten shows ground shaking, and a sea-level jump is rare (1.5%). That is why a demo produces many flood alerts and very few tsunami alerts. It is intentional: it lets the alarm and the screens be tested quickly.

> **IN SIMPLE WORDS**
> The simulator is like a video-game weather machine: mostly sunny, with a storm dropped in from time to time so you can see the alarm work.

6.2 Switching to real hardware

A single switch in the settings, `USE_REAL_HARDWARE`, decides whether the program reads simulated numbers or real sensors. The real-sensor code is already written as commented steps in `reader.py` and `dispatcher.py`, ready for a Raspberry Pi.

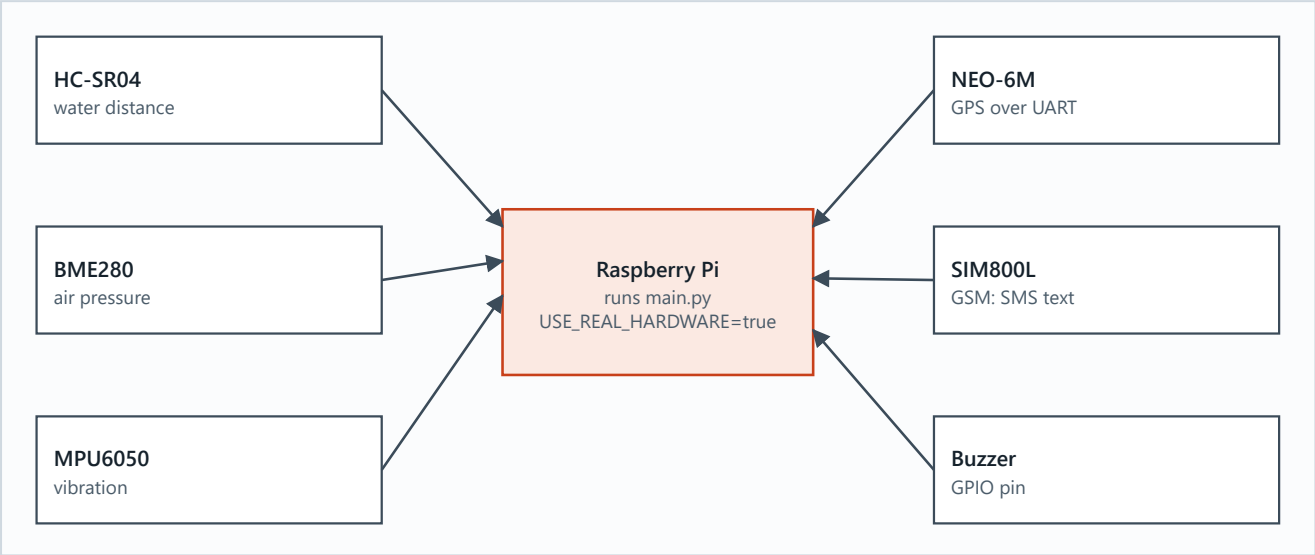


Figure 10. The planned hardware. Only the Pi and its connections would change; every detector stays exactly the same.

Table 10. Planned real parts and how the Pi talks to them.

PART	MEASURES	HOW IT CONNECTS
HC-SR04	Distance down to water	Two GPIO pins (trigger and echo)
BME280	Pressure, also temperature	I^2C bus
MPU6050	Acceleration (shaking)	I^2C bus
NEO-6M	Position from satellites	Serial `/dev/ttyS0`, 9600 baud
SIM800L	Sends SMS with no internet	Serial `/dev/ttyUSB0`, AT commands
Buzzer	Makes the alarm sound	One GPIO output pin

WHY WE CHOSE THIS

Fail loudly, not silently. If the hardware switch is turned on before the real code is filled in, the program raises `NotImplementedError` instead of quietly returning fake numbers. A silent fake reading in a safety system is worse than a crash.

CHAPTER 7

The four detectors

The detectors are the “brain” of the project. Each is a short Python file with a clear yes/no rule. This chapter opens each one up.

7.1 Flood detector

File: `agents/flood_agent.py`. It checks up to three things. If *any* is true, the risk is **high**, and every true reason is listed in the message.

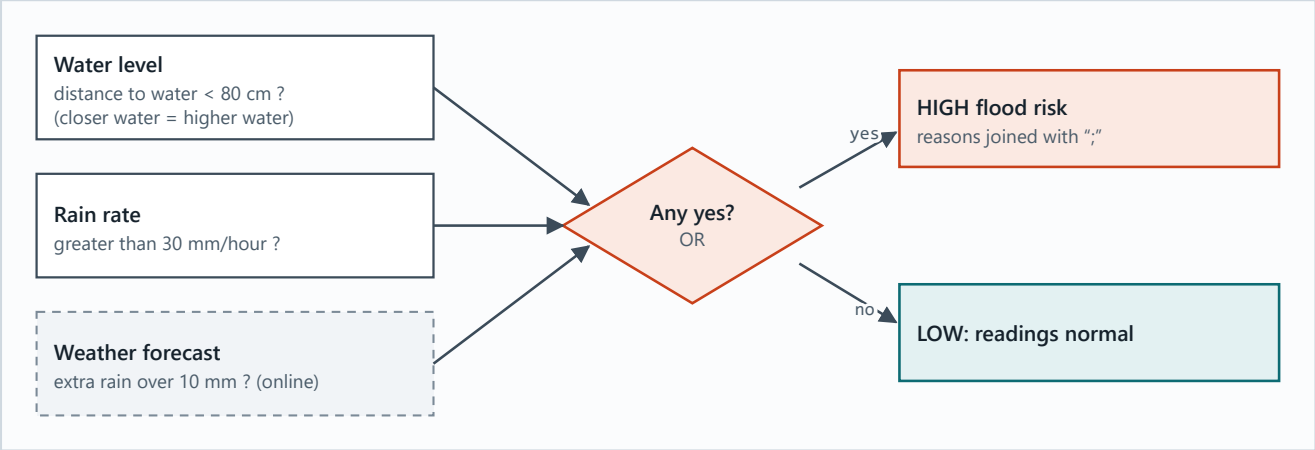


Figure 11. The flood rule. Three checks feed an OR gate. The third check is an extra that only exists when online.

Table 11. Flood limits and the words used in alerts.

CHECK	SETTING NAME	LIMIT	MESSAGE
Water distance	`FLOOD_WATER_LEVEL_CM`	`< 80`	“water level high (… cm from sensor)”
Rain rate	`FLOOD_RAIN_RATE_MM_HR`	`> 30`	“heavy rain (… mm/hr)”
Forecast rain	built in	`> 10 mm`	“external forecast also shows heavy precipitation”

IN SIMPLE WORDS

Picture a bucket under a leaking roof. If the water is already near the top, or it is raining very hard, or the weather report says more is coming, you call for help.

WHY WE CHOSE THIS

Plain thresholds. There is no model of rivers or terrain, because that needs elevation data the project does not have. A simple limit is honest about what it knows. A note in the code says the same about the 2 km flood circle drawn on maps: it is a fixed illustration, not a flood-extent forecast.

Unused setting. `FLOOD_PRESSURE_DROP_HPA = 6` exists in the settings but no detector reads it yet. Pressure is recorded and displayed for future use.

7.2 Earthquake detector: STA/LTA

File: `agents/earthquake_agent.py`. The ground always trembles a little, so a fixed limit is a poor test. Seismologists use **STA/LTA**: the *short-term average* of the shaking divided by its *long-term average*. In calm times the ratio is near 1. A sudden earthquake makes the short average jump, so the ratio climbs.

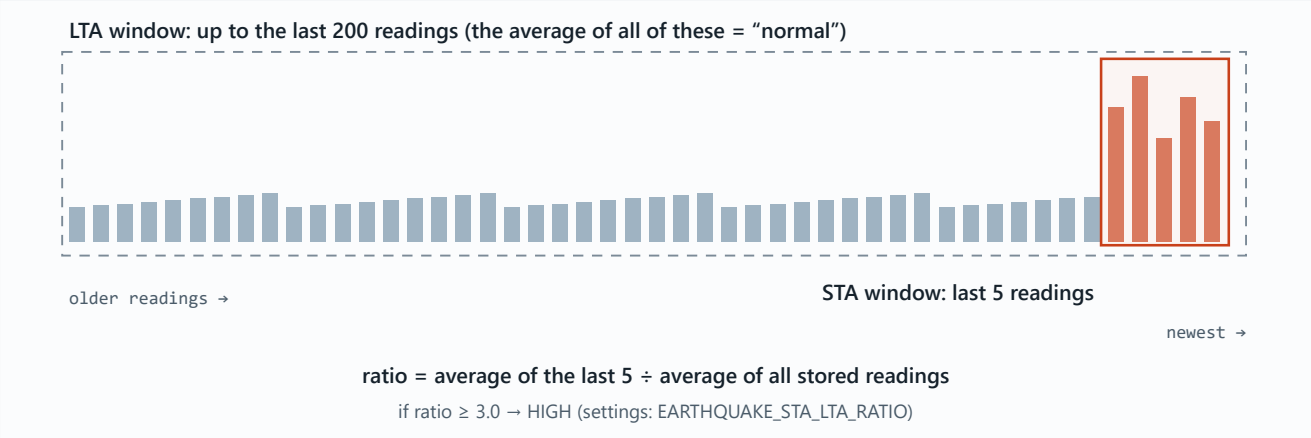


Figure 12. The two windows. Grey bars are quiet readings; red bars are a burst of shaking inside the short window.

A worked example (the same maths the manual test uses)

Table 12. Twenty readings: nineteen quiet (0.01 g) and then one new reading.

NEW READING	LONG AVERAGE	SHORT AVERAGE (LAST 5)	RATIO	VERDICT
`0.02 g`	`0.0105`	`0.012`	`1.14`	low
`0.30 g`	`0.0245`	`0.068`	`2.78`	low
`0.50 g`	`0.0345`	`0.108`	`3.13`	**HIGH**

- The live check keeps a rolling list of the last **200** readings and needs at least **6** before it will give a ratio (otherwise it reports 0.0).
- If a *historical* event is supplied (used by the replay test), the detector returns high at once and quotes the magnitude and place.
- A **manual test** uses `check_earthquake_risk_isolated`, which compares the value against 19 quiet readings so results never depend on earlier tests.

> **WHY WE CHOSE THIS**
>
> **STA/LTA over a fixed number.** It adapts to how noisy a place normally is, needs no training data, runs in microseconds, and is a technique real seismic networks use.

7.3 Storm detector

File: `agents/storm_agent.py`. Three rules are checked in order. A thunderstorm code from the weather feed is enough on its own to raise “high”, even if local sensors are quiet. Otherwise strong wind or extreme rain does it.

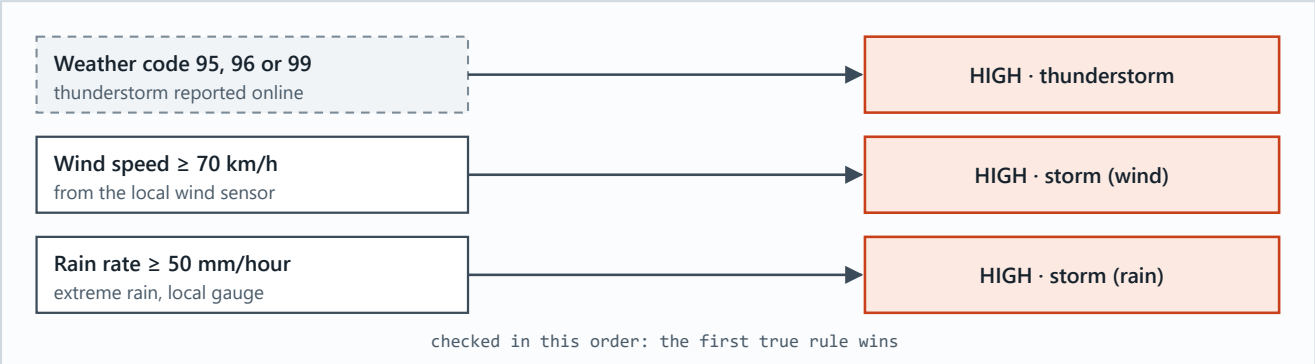


Figure 13. The three storm rules, checked top to bottom.

7.4 Tsunami detector

File: `agents/tsunami_agent.py`. It takes the size of the sea-level change, `abs(sea_level_change_cm)`, and compares it with **50 cm**. Using the absolute value means a fast fall (the sea pulling back) is treated the same as a fast rise. The file says in words that a river or pond reading is *not* valid evidence.

7.5 Combining the answers

Only one headline is shown, chosen in order of how little time people have to react.

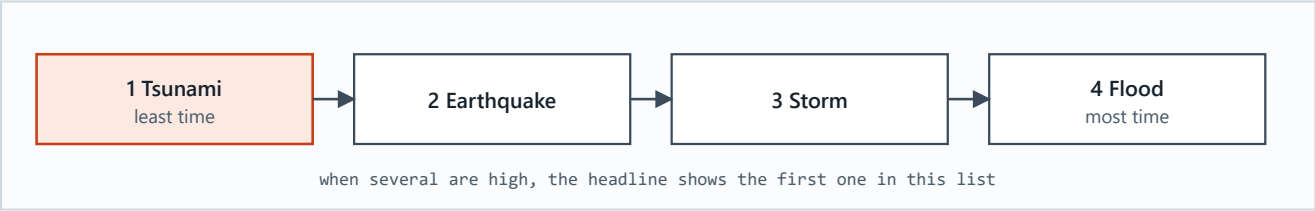


Figure 14. Priority order used by `combine_risk`.

WHY WE CHOSE THIS

Worst wins. Showing one headline keeps the alert short and clear. The individual answers for all four hazards are still returned and displayed on the Home screen, so no information is lost.

CHAPTER 8

Working with and without the internet

This is the central idea of the project. The alarm should never need the network, but should use it happily when it is there.

8.1 Three modes

A setting called `DISASTER_APP_MODE` (or the `--mode` flag) chooses how the program decides whether it is online.

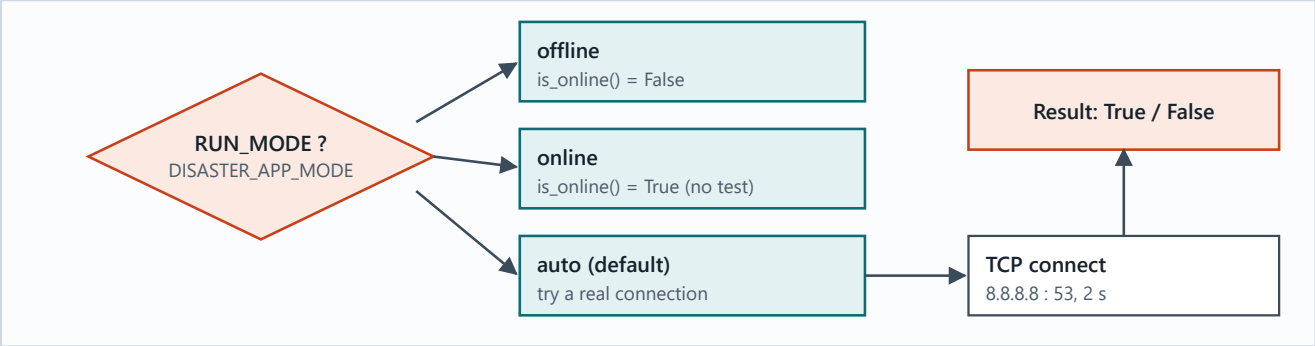


Figure 15. How `is_online()` works in each mode.

Table 13. The three modes.

MODE	BEHAVIOUR	USE IT FOR
auto	Checks the connection on every cycle	Normal running
offline	Never touches the network	Testing, tests in CI, demos with no Wi-Fi
online	Assumes a connection without testing	Forcing the online path

8.2 The connection test

Instead of downloading a web page, the program opens a raw network connection to Google's public DNS server (`8.8.8.8`, port 53) and gives up after 2 seconds. Nothing is sent. A connection either works or it does not.

WHY WE CHOSE THIS

A socket, not a web request. A page download is slow and can fail for reasons unrelated to being online. A raw connection takes a few hundred milliseconds when online and fails quickly offline, so it is cheap enough to run on every 5-second cycle.

IN SIMPLE WORDS

It is like knocking on a friend's door to see if anybody is home, without going inside.

Honest detail. In `online` mode the function returns True without testing. That is fine for forcing the path in a test, but the mode should not be used in the field.

8.3 Saving while offline, catching up later

Every reading is written to the database *before* anything else. If the internet is present, the row is immediately marked `synced = 1`. If not, it stays at `synced = 0`. The moment the program sees the connection return, it walks through all unsynced rows and marks them.

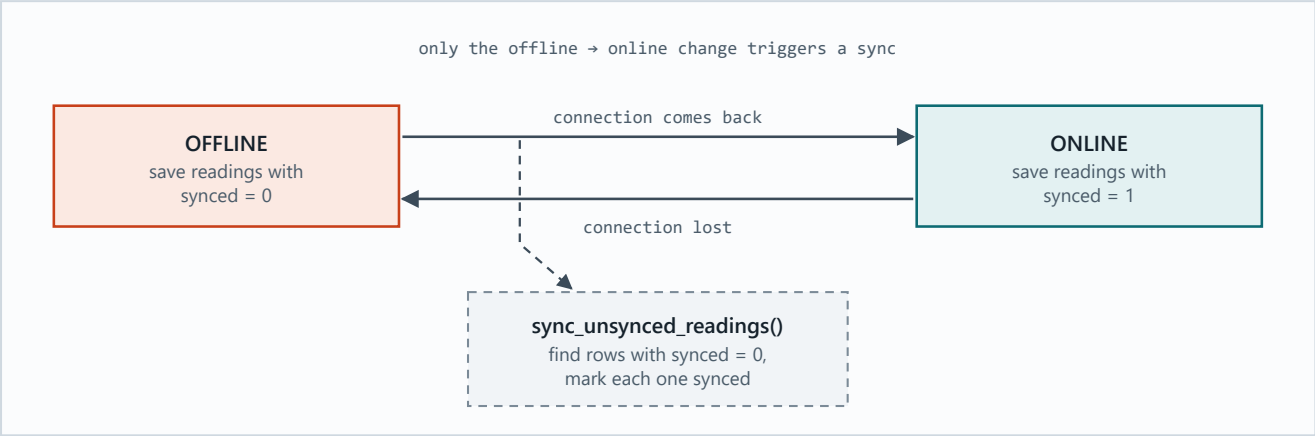


Figure 16. Two states. Only the change from offline to online triggers a catch-up.

> **Honest detail.** The upload itself is a commented-out stub in `main.py` (`requests.post(...)`). Today the catch-up only marks rows as synced. A real cloud server would slot in at that one line.

8.4 Every online call is built to fail safely

Table 14. Network helpers, their time limit and what they return when something goes wrong.

HELPER	TALKS TO	TIME LIMIT	ON FAILURE
`fetch_weather_context`	Open-Meteo	`3 s`	returns `None`
`find_nearby_shelters`	OpenStreetMap Overpass	`12 s`	returns `[]`
`_online_events`	USGS earthquake catalog	`8 s`	returns `[]` (offline catalog still used)
`explain` node	Groq language model	`4 s`	keeps the rule sentence

> WHY WE CHOSE THIS
>
> **Fail safe, never fail loud.** If every helper caught its own errors and returned an empty value, the calling code needs no special cases. There is one rule everywhere: “if you get nothing, carry on with what you have.”

CHAPTER 9

Alerts and storage

9.1 The alert dispatcher

File: `alerts/dispatcher.py`. When the combined result is **high**, the dispatcher builds one message and sends it down several channels. When it is low, the function returns immediately.

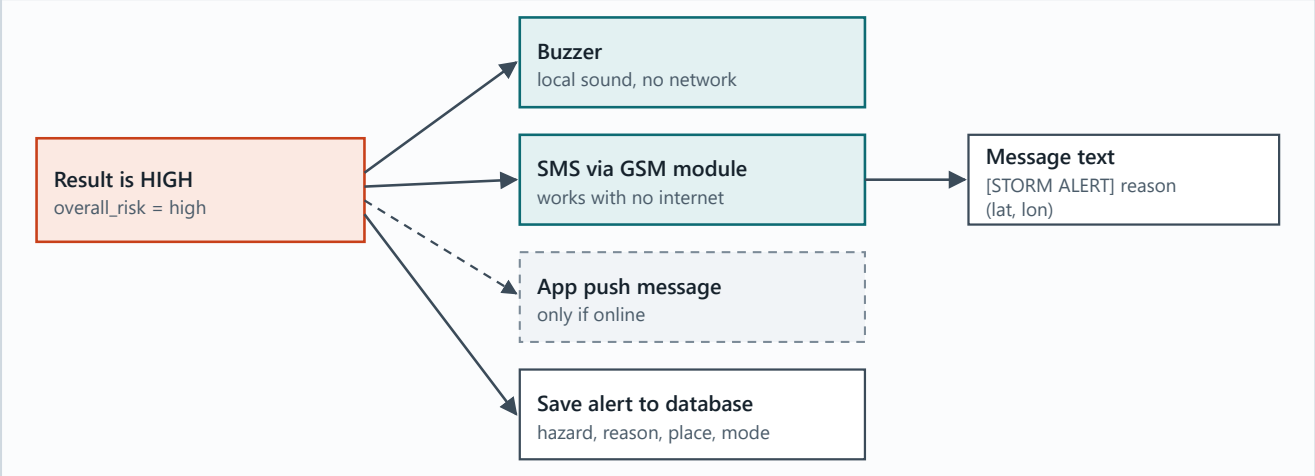


Figure 17. Alert channels. Solid teal boxes are guaranteed; the dashed box is a bonus.

Guaranteed and bonus channels

Table 15. The four things that happen on a high alert.

CHANNEL	NEEDS INTERNET?	WHAT IT DOES TODAY
Buzzer	No	Prints “Buzzer triggered.” (GPIO code ready for hardware)
SMS	No	Prints the text and contact list (AT-command code ready for a SIM800L)
App push	Yes	Prints the message (bonus channel)
Database	No	Really saves the alert with time, place and mode

WHY WE CHOSE THIS

Sound and text first. A siren and an SMS reach the most people with the least dependence on other systems. App push is nicer but needs the network, so it can only ever be an addition.

Two honest details. The SMS contact list has one default number written into the code (`+917620622761`); it should be moved to the settings file. And because no real GSM hardware is connected, the SMS and buzzer channels only *print* what they would do.

9.2 The database

File: `database/db.py`. All memory lives in one SQLite file, `data/disaster_alert.db`, with two tables.



Figure 18. The two tables. In our test the file held more than 585 readings and 121 alerts in only about 94 KB.

Table 16. The helper functions.

FUNCTION	PURPOSE
`init_db()`	Create both tables if missing; add newer columns to an old file automatically
`save_reading(r)`	Insert one sensor snapshot; returns its id
`save_alert(…)`	Insert one alert
`recent_readings(n)` / `recent_alerts(n)`	Newest rows first, for the charts and the Report screen
`unsynced_readings()` / `mark_synced(id)`	Support the offline catch-up

WHY WE CHOSE THIS

SQLite. It needs no server, no password and no install: a whole database is one file that ships inside the zip. It is fast for this size, safe to write on every cycle, and understood by every tool.

IN SIMPLE WORDS

A database is a very tidy notebook. SQLite is a notebook that fits in a single file you can copy to a USB stick.

All queries use *parameters* (`?` placeholders) instead of pasting values into the text. This is the standard protection against SQL injection.

Automatic upgrade. When storm and tsunami were added, two new columns were needed. `init_db()` checks the table's columns and adds `wind_speed_kmh` and `sea_level_change_cm` if they are missing, so older database files keep working.

CHAPTER 10

The backend server

File: `backend/server.py`. The server is the waiter between the brain and the screens. The screens ask questions in a fixed format (called an **API**) and the server answers with structured data (JSON).

10.1 The endpoints

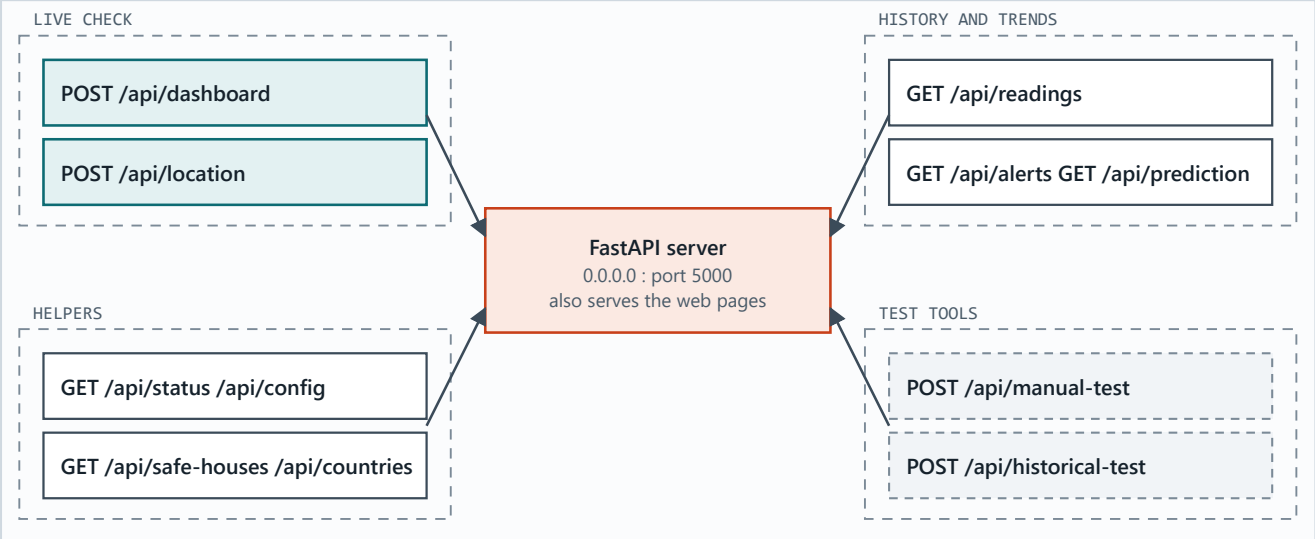


Figure 19. The eleven endpoints, grouped by purpose. The dashed boxes are developer test tools.

Table 17. Every endpoint and what it returns.

ADDRESS	RETURNS
`GET /api/status`	Whether the server thinks it is online
`GET /api/config`	All alert limits, so the screen can never drift from the rules
`GET /api/readings, /api/alerts`	Newest stored readings and alerts
`GET /api/safe-houses`	Nearby shelters (widens the search once if none are found)
`GET /api/prediction`	A short trend estimate from recent readings
`POST /api/dashboard`	One live check in the compact form the phone screens use
`POST /api/location`	One live check returning the full detector result
`POST /api/manual-test`	A check on hand-typed sensor values (now includes wind and sea level)
`POST /api/historical-test`	Replay a past earthquake by country and date

10.2 The trend estimate (“Prediction” screen)

The Prediction screen does not use artificial intelligence. It fits a **straight line** through the last few readings (a least-squares slope) and extends it three readings ahead. If that projected value crosses the flood limit, the screen shows a caution.

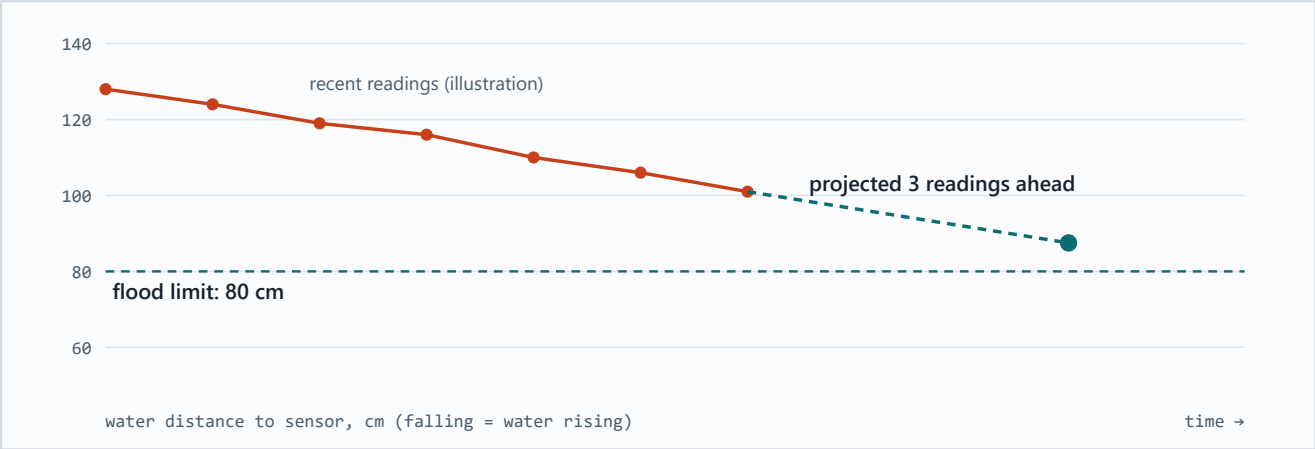


Figure 20. How the projection works. The red line is recent water distance, the dashed teal line is the straight-line projection, and the dashed limit is 80 cm.

WHY WE CHOSE THIS

A line, not a neural network. With only a few local readings, a straight line is honest and easy to explain. A larger model would appear more clever but would only be guessing, and it could not show its working.

Not an earthquake predictor. Earthquakes cannot be reliably predicted from a short local vibration history. The vibration panel is labelled “activity monitoring” and only flags when the projected shaking passes 0.1 g.

10.3 Other server details

- **Checked inputs.** Incoming data is described by Pydantic models (`LocationIn`, `ManualTestIn`, `HistoricalTestIn`). A request with a missing or wrong-typed field is refused automatically with an error message.
- **Shelter search.** If nothing is found within 5 km, the server retries once at three times the radius, up to 20 km.
- **Web pages.** The `frontend` folder is mounted at “/” *after* all API routes, so an address like `/api/status` always reaches the API first.
- **Startup.** Running `python -m backend.server` starts the server on port 5000 and opens the browser.

CHAPTER 11

The user app

Files: `frontend/index.html`, `mobile.css` and `mobile.js`. The app looks and behaves like a phone app but runs in any browser. A navigation bar along the bottom switches between five screens.

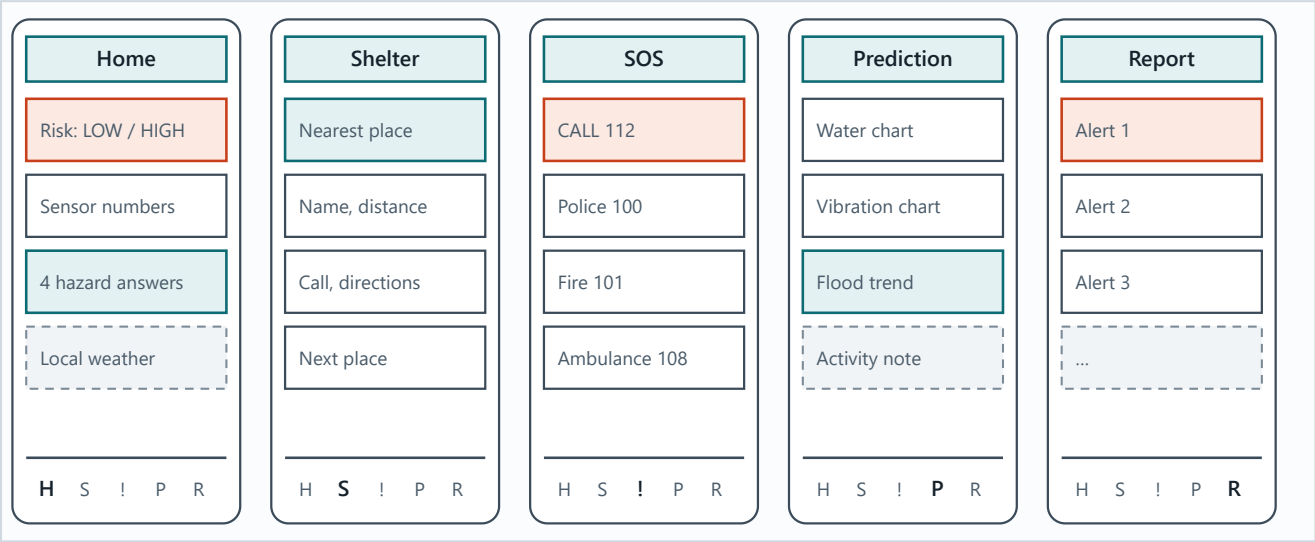


Figure 21. The five screens. The bottom letters are the navigation bar: Home, Shelter, SOS, Prediction, Report.

Table 18. What each screen shows and which server address it uses.

SCREEN	PURPOSE	SERVER CALL
Home	Current risk, each hazard, sensor numbers, weather	`POST /api/dashboard`
Shelter	Nearest safe places with distance, call and directions	`GET /api/safe-houses`
SOS	One big call button and emergency numbers	none (works offline)
Prediction	Charts and the trend estimate	`GET /api/prediction`
Report	History of past alerts	`GET /api/alerts`

> **IN SIMPLE WORDS**
> The SOS screen needs no internet at all, because the phone can place a call by itself. In an emergency, the most important button should be the one that always works.

11.1 How the app finds where you are

The whole point of the app is "what is the risk *here*?", so knowing the location is key. The app asks the browser for the phone's position. If that is refused or fails, it falls back to the last position it saved. It can also be told a place in the address bar with `?lat=…&lon=…`, which is handy for demos.

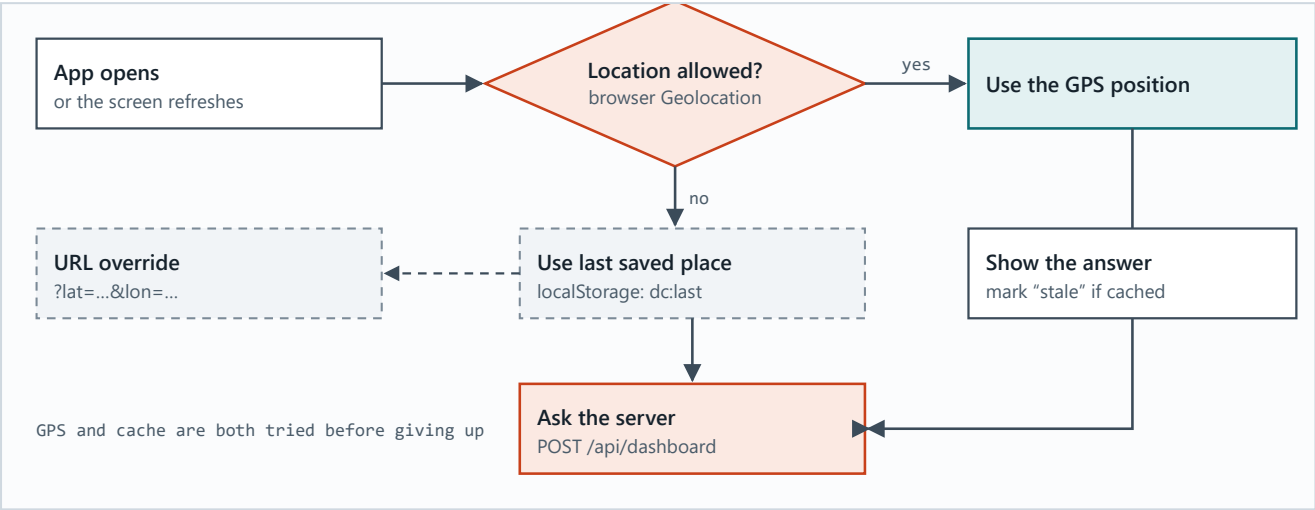


Figure 22. Location is tried from three sources before the app gives up.

11.2 Choices in the app code

- **No framework.** Plain HTML, CSS and JavaScript. There is nothing to build or install, and the files are tiny.
- **Safe display.** Server or map text is always inserted with `textContent`, never as raw HTML, so a malicious shelter name cannot inject code.
- **Remembers the last answer.** The last result and shelter list are saved in the browser (`localStorage`), so a weak connection still shows something useful, marked "stale".
- **Distance in the browser.** A **haversine** formula computes the distance between two points on Earth, so shelters can be sorted even from cache.
- **Alerts while open.** A bell button asks for permission and shows a browser notification when the risk becomes high. There is no background push service.
- **Ready-made actions.** Shelters have call and directions links; the SOS screen dials 112 (police 100, fire 101, ambulance 108), and the numbers are easy to edit for another country.

> **WHY WE CHOSE THIS**
> **Vanilla JavaScript.** A framework such as React would add a build step and hundreds of files to a project that has five small screens. Plain code loads instantly, works offline from the zip, and can be read in one sitting.

CHAPTER 12

The A–Z register: everything we used, and why

This chapter is the reference list. Every tool, library and idea used in the project has an entry, in alphabetical order. Each entry answers three questions: **what** is it, **why** did we use it, and what did we choose it **instead of**.

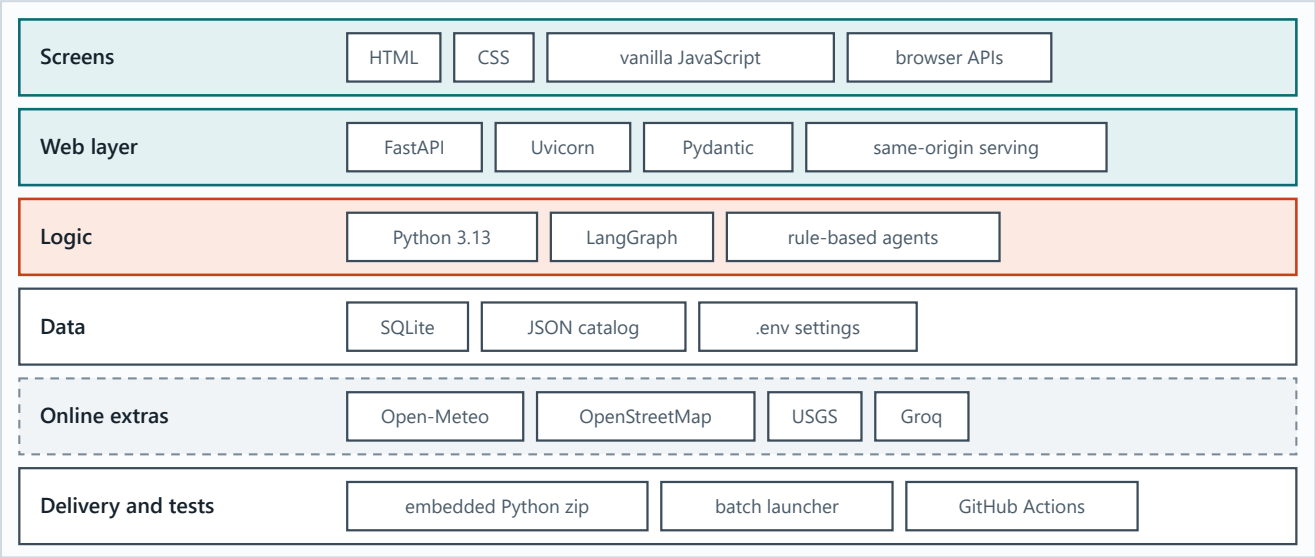


Figure 23. The whole technology stack in six layers. Everything above "Online extras" works with no internet.

A

Agents (rule-based detectors)

WHAT Four small Python files, one per hazard, each returning "low" or "high" plus a reason.

WHY Short rules can be read, checked and tuned by a person, and they run instantly with no network.

INSTEAD OF A trained machine-learning model, which needs real disaster data we do not have and cannot explain itself.

B

Browser APIs (Geolocation, Notifications, localStorage)

WHAT Features built into every modern browser: position, pop-up alerts, and small local storage.

WHY They give the web app phone-like abilities with zero extra software.

INSTEAD OF A native mobile app, which needs separate builds for each phone type.

C

Continuous integration (GitHub Actions)

WHAT A workflow file that automatically runs the checks on every code push.

WHY Catches a broken change within minutes: it runs the loop offline, starts the server and checks its addresses, and imports every module.

INSTEAD OF Remembering to test by hand before each change.

D

python-dotenv and the `.env` file

WHAT A small library that loads secret settings from a private file into the program.

WHY Keeps the Groq key out of the code and out of the shared zip.

INSTEAD OF Typing keys straight into source files, where they leak easily.

E

Earthquake catalog (USGS + bundled JSON)

WHAT A live list of past earthquakes from the US Geological Survey, plus a small offline copy of famous ones.

WHY Lets the test tool replay a real event by country and date, even with no internet.

INSTEAD OF Only a live feed, which fails offline, or only a fixed list, which cannot find new events.

F

FastAPI

WHAT A modern Python web framework for building an API.

WHY It is fast, checks incoming data automatically with Pydantic, and writes its own documentation page. The whole server is one readable file.

INSTEAD OF Flask (older docs of this project mention it), which needs extra tools for the same data checking.

G

Groq (language model service)

WHAT An online service that runs a language model. We ask it for one plain sentence about an alert.

WHY Makes the alert easier for the public to understand. It is optional and wrapped in a 4-second timeout.

INSTEAD OF Letting the model decide risk, which is unsafe, or leaving the wording technical.

H

Haversine formula

WHAT A maths formula that gives the distance between two latitude/longitude points on a sphere.

WHY Used to find which shelters are nearest and to reject far-away earthquakes. It is a few lines, needs no library and runs offline.

INSTEAD OF A mapping library, which is heavy and needs the internet.

I

Internet check (raw socket)

WHAT A quick connection to `8.8.8.8:53` with a 2-second limit.

WHY Tells us whether the network is up, cheaply, on every cycle.

INSTEAD OF Downloading a web page, which is slower and can fail for other reasons.

J

JSON

WHAT A plain-text format for structured data, like {"risk": "low"}.

WHY The server, the screens, the earthquake catalog and the weather service all speak it, so nothing needs converting.

INSTEAD OF XML or custom text, which are longer and harder to read.

K

Keys and secrets

WHAT The Groq API key, held only in `.env` and read as `GROQ_API_KEY`.

WHY A secret must never travel with the code. The packaging script deliberately leaves `.env` out of the zip.

INSTEAD OF Hard-coding the key, which would leak it to everyone who receives the zip.

L

LangGraph

WHAT A library that runs steps in order while passing one shared dictionary between them.

WHY It makes the chain of detectors explicit and lets branches or parallel steps be added later.

INSTEAD OF Plain function calls, which work today but grow untidy if the flow becomes complex.

M

Mobile-first vanilla JavaScript

WHAT The app screens written in plain JavaScript for phone-sized displays.

WHY No build step, tiny files, instant load, works offline from the zip.

INSTEAD OF React or Vue, which add tools and files a five-screen app does not need.

N

Network fail-safe pattern

WHAT Every internet helper catches its own errors and returns `None` or an empty list.

WHY A dead connection lowers the extras but never the alarm, and callers need no special cases.

INSTEAD OF Letting errors bubble up, which could crash the loop at the worst moment.

O

Open data services (Open-Meteo, OpenStreetMap)

WHAT Open-Meteo gives current weather; OpenStreetMap's Overpass service lists hospitals, community centres and shelters.

WHY Both are free and need no API key, and their data is good enough for a demo.

INSTEAD OF Paid weather and map APIs, which need accounts and billing.

P

Python 3.13

WHAT The programming language of the whole back end.

WHY Very readable, strong libraries for web and data work, and it runs on both a laptop and a Raspberry Pi.

INSTEAD OF Java or C++, which are slower to write and heavier for a small team.

Q

Query parameters (SQL placeholders)

WHAT Database commands written with `?` holes filled in separately.

WHY Stops SQL injection, where bad text could change the meaning of a command.

INSTEAD OF Building command text by gluing strings together.

R

requests library

WHAT A friendly Python library for calling web services.

WHY Handles timeouts and errors cleanly for the weather, USGS and shelter calls.

INSTEAD OF The lower-level built-in modules, which need far more code.

S

SQLite

WHAT A complete database that lives in one file.

WHY No server, no password, no install; it ships inside the zip and handles this data with ease.

INSTEAD OF MySQL or PostgreSQL, which need a running database server.

T — Thresholds and STA/LTA

WHAT Fixed limits in `settings.py` and, for earthquakes, the short-average ÷ long-average ratio of shaking.

WHY Limits are transparent and tunable in one place. STA/LTA adapts to how noisy a place normally is and is a classic seismology method.

INSTEAD OF One fixed vibration number, which false-alarms in noisy places and misses quakes in quiet ones.

U — Uvicorn

WHAT The engine that actually runs a FastAPI app and listens on a network port.

WHY It is the standard, lightweight partner for FastAPI and starts in a moment.

INSTEAD OF A heavier production server, unneeded for a local tool.

V — Virtual environment (venv)

WHAT A private folder of libraries for one project.

WHY Keeps this project's versions from clashing with others. The setup scripts use one.

INSTEAD OF Installing everything system-wide, which becomes messy fast.

W — Windows batch launcher (`START.bat`)

WHAT A tiny script that starts the server using the bundled Python.

WHY A person who cannot use a terminal can still start the app with a double-click.

INSTEAD OF Asking users to open a command window and type commands.

X — XSS-safe rendering

WHAT Showing server text with `textContent`, not as HTML.

WHY Prevents cross-site scripting: text that pretends to be code cannot run inside the page.

INSTEAD OF Building screens with `innerHTML` on untrusted text.

Y — YAML workflow file (`ci.yml`)

WHAT A readable settings file that tells GitHub what to test.

WHY Three jobs: core pipeline offline, server health check, and module imports.

INSTEAD OF A script kept only on one person's laptop.

Z — Zip packaging with embedded Python

WHAT `build_portable.py` builds one zip holding the code, all libraries and a copy of Python.

WHY The receiver needs to install nothing. That was the "zip and it runs" goal.

INSTEAD OF A normal installer or asking people to install Python first, which is where most people give up.

> **IN SIMPLE WORDS**
>
> The A–Z list shows a pattern. Each choice picks the smaller, simpler tool that still does the job, and prefers things that keep working when the internet does not.

CHAPTER 13

Packaging: from a folder to “just double-click”

Most programs fail on someone else’s computer because they need other software installed first. We removed that problem by putting everything in one folder.

13.1 What is inside the zip

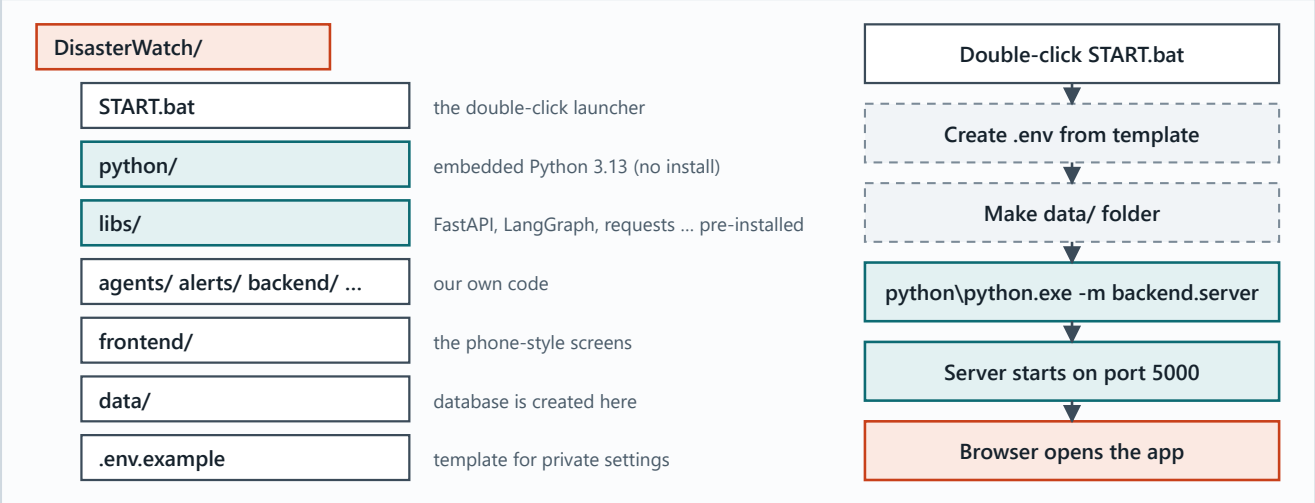


Figure 24. Left: the folders in `DisasterWatch.zip`. Right: what happens when `START.bat` is double-clicked.

13.2 The trick that makes it portable

Python normally lives in the system and finds its libraries there. The **embeddable** Python package from python.org is a tiny, self-contained copy that can sit inside any folder. It ignores the computer’s own settings, so a small file next to it, `python313._pth`, lists exactly where to look: the `libs` folder and the folder above it.

WHY WE CHOSE THIS

Bundle Python itself. A first version of the launcher created a virtual environment and downloaded libraries on the first run. That still needed Python installed and the internet. Bundling both removed the last two things a user could get wrong.

IN SIMPLE WORDS

It is like sending someone a meal kit with the pan and the stove included, instead of a recipe and a shopping list.

13.3 How the zip is built

The script `build_portable.py` repeats six steps every time it is run. That makes the build repeatable and removes the risk of forgetting a file.

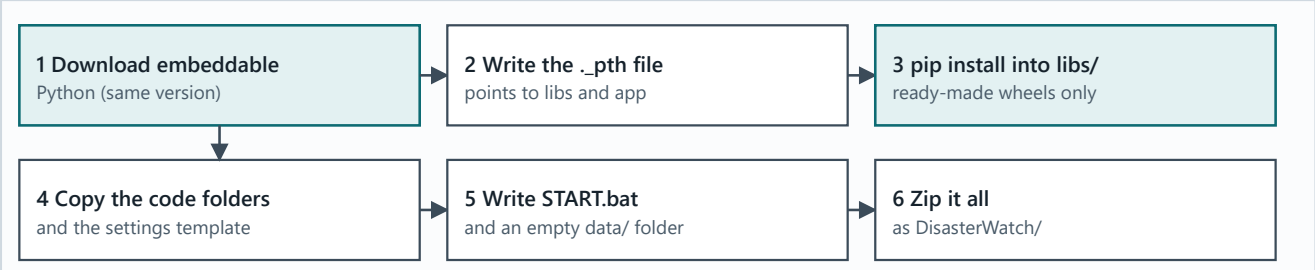


Figure 25. The six steps of `build_portable.py`.

Deliberately left out: the private `.env` file (it holds the Groq key), the developer's database, the virtual environment and caches. The result is about **28 MB**.

13.4 Other ways we considered

Table 19. Delivery options that were tried or weighed, and what happened.

OPTION	GOOD	PROBLEM	OUTCOME
Zip + first-run installer	Small download	Needs Python and internet	Replaced
Zip with bundled Python	Nothing to install	Windows 64-bit only, 28 MB	**Chosen**
Single .exe (PyInstaller)	One file	Web servers and LangGraph hide imports; harder to debug	Not pursued
Android app (Capacitor)	Installs like a phone app	The Python server must run somewhere the phone can reach	Built as a trial, then removed
Cloud hosting	Works from anywhere	Needs an account; free tiers sleep and lose data	Prepared, not used

13.5 Running from the source code

Table 20. The ways to run the project during development.

COMMAND	RESULT
`python main.py`	Monitoring loop in the terminal (auto mode)
`python main.py --mode offline --once`	One offline cycle, then exit (what CI runs)
`python -m backend.server`	Web app at http://localhost:5000
`python build_portable.py`	Builds `dist/DisasterWatch.zip`

CHAPTER 14

Testing and results

14.1 Strategy

The tests are layered like a pyramid: many quick, cheap checks at the bottom that catch simple breaks, and fewer, slower checks at the top that prove the whole thing works.

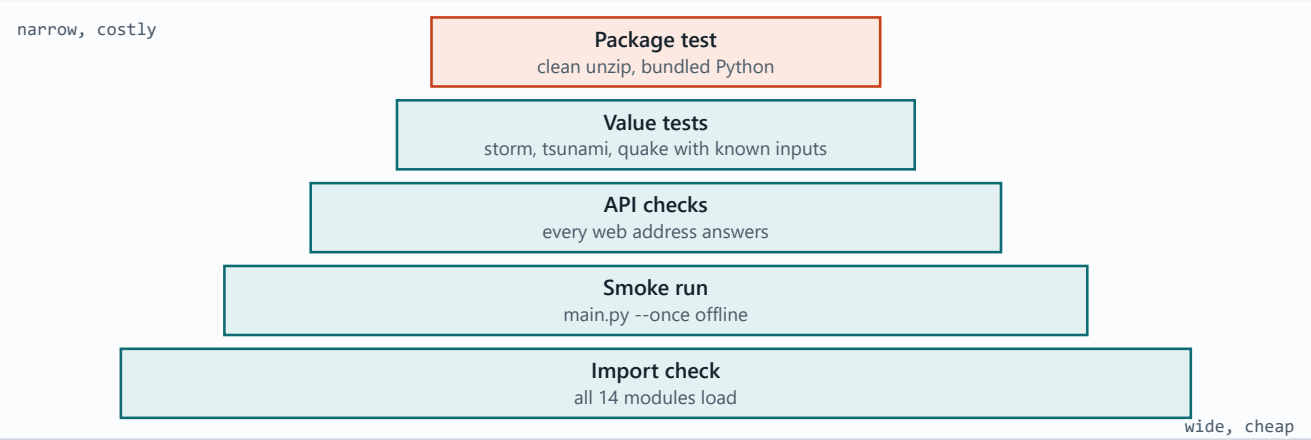


Figure 26. Five levels of testing, cheapest at the bottom.

Table 21. Checks that were run on 20 September 2026, and their results.

CHECK	METHOD	RESULT
Module imports	Imported all 14 modules	**Pass** — 14 of 14
One full cycle	`main.py --once`, offline	**Pass** — printed a risk line
Web server	Called status, config, readings, alerts, home page	**Pass** — all answered
Live dashboard	`POST /api/dashboard` for Pune	**Pass** — risk, sensors, real weather
Storm detector	Wind 120 km/h; rain 200 mm/hr	**Pass** — “high · storm” both times
Tsunami detector	Sea-level change 300 cm	**Pass** — “high · tsunami”
Portable zip	Unzipped elsewhere, bundled Python only	**Pass** — ran; database created

14.2 Automatic checks (GitHub Actions)

The file `.github/workflows/ci.yml` tells GitHub to repeat the basic checks on every push, so a broken change is noticed at once.

Table 22. The three CI jobs.

JOB	WHAT IT DOES
Core pipeline	Installs the libraries and runs `main.py --once` in offline mode
Server health	Starts the server, then calls `/api/status`, `/api/readings` and `/api/alerts`
Module imports	Imports the core modules one by one to catch a typing or import mistake

14.3 A worked example

We sent this test to the manual-test address, giving a very high wind and calm values for everything else:

```
water_level_cm = 150   rain_rate_mm_hr = 1   pressure_hpa = 1010
vibration_g = 0.01   wind_speed_kmh = 120   sea_level_change_cm = 0
```

The answer came back as **high · storm** with the reason *"wind speed 120.0 km/h exceeds trigger (70 km/h)"*. The buzzer and SMS lines were printed, and an alert row was saved. Every part of the chain worked together.

14.4 Problems found, and how they were fixed

Table 23. Bugs and loose ends discovered while testing.

PROBLEM	CAUSE	FIX
Storm and tsunami "not working"	The running server had started before those files were added	Restart with the current code
Manual test could not trigger storm or tsunami	The input form dropped wind and sea-level values	Added both fields to `ManualTestIn`
Imports failed on a fresh Python	Libraries were not installed	Bundled in the zip
Unused desktop dashboard and scripts	Left over from an earlier UI	Removed
CI check expects empty history	The test looks for `[]`, which fails once real rows exist	Noted; fine on a fresh database
Some older documents out of date	They describe only flood and earthquake	Noted for update

14.5 What was not tested

- Real sensors, a real SIM800L module and a real buzzer (only simulated).
- Long-running stability over many hours or days.
- Other phones and browsers, and non-Windows computers for the zip.
- The GitHub Actions workflow itself, because the project was never pushed to GitHub. Its three jobs were read and their commands were run by hand instead.
- Heavy load or many users at once.

> **IN SIMPLE WORDS**
> Testing a smoke alarm on a real fire is the only full proof. What we did was press its test button many ways: everything that could be checked without real hardware passed.

CHAPTER 15

Security, privacy and honest limits

15.1 What leaves the computer

Location is the most sensitive thing the app handles. Figure 27 shows every outside service and exactly what it receives. Everything else stays on the computer.

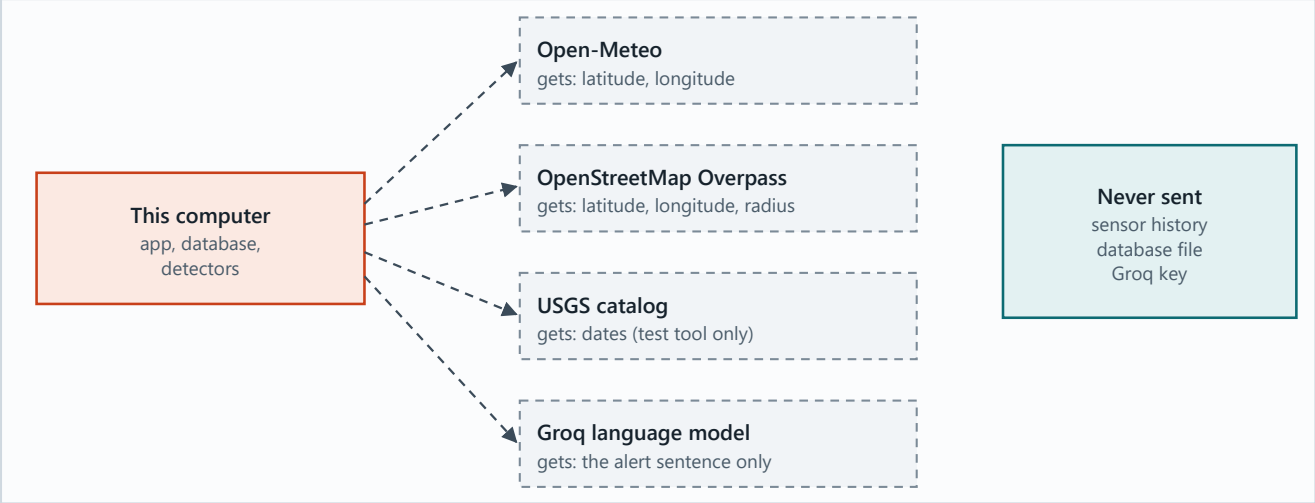


Figure 27. Data that leaves the computer, and only when online. Dashed arrows are optional.

Table 24. Security measures and known gaps.

TOPIC	STATUS
Secrets	Groq key lives in `.env`; never packaged
SQL injection	Prevented with placeholders
Script injection (XSS)	Prevented with `textContent`
Input checking	Pydantic refuses badly formed requests
Login	**None.** The server listens on all network addresses; keep it on a trusted network
Hard-coded SMS number	One default contact sits in `dispatcher.py`; move to settings

WHY WE CHOSE THIS

Send as little as possible. Each online helper sends only what it needs. The language model never sees the location, and no service ever receives sensor history.

CHAPTER 16

Future work and conclusion

16.1 What comes next

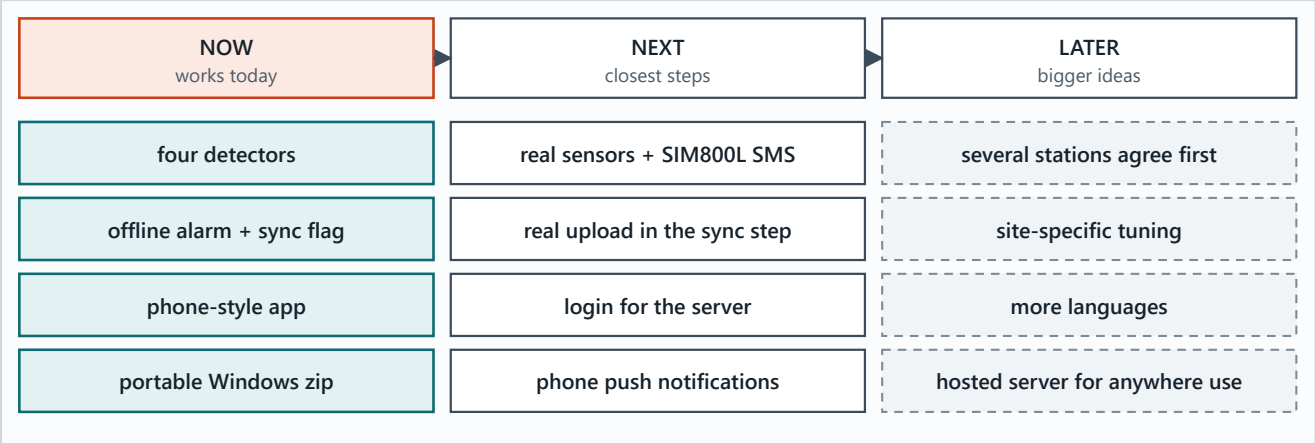


Figure 28. A three-step roadmap. “Now” is what this report describes.

- **Real hardware.** Fill in the commented sensor code, connect a SIM800L and a buzzer, and tune every limit against a real site.
- **Fewer false alarms.** Ask nearby stations to agree, and require a reading to stay high for a short time before alerting.
- **Real reach.** A real upload in the sync step, push notifications when the app is closed, and translations of the app.
- **Use the pressure reading.** A sudden pressure drop often comes before a storm; the setting for it already exists.

16.2 Conclusion

Disaster Connect shows that a useful warning system can be simple. Four short rule files decide, a single loop runs them every five seconds, and one SQLite file remembers the past. The internet makes the answers richer but is never in the way of the alarm. The system is explained by its own reasons, tested at five levels, and delivered as one zip that runs without installing anything.

16.3 What we learned

- Build the safe path first, then add extras around it.
- Prefer readable rules to clever models when a wrong answer is costly.
- Make failure boring: every online call returns nothing instead of crashing.
- A tool nobody can start is a tool nobody uses, so packaging is part of the design.

> **Reminder.** This is a learning and demonstration project, not an official warning service. Nobody should rely on it to stay safe.

APPENDIX

Glossary and references

API — A menu of questions one program can ask another.

Backend — The server-side program that does the work.

Detector (agent) — A small rule-checker for one hazard.

Endpoint — One address on the server, such as /api/status.

Fail-safe — Designed so a failure of an extra does not stop the essential job.

False alarm — An alert when there is no real danger.

GPS — Satellite positioning that gives latitude and longitude.

Haversine — Formula for distance between two points on Earth.

JSON — A plain-text way to write structured data.

LTA / STA — Long-term and short-term average of a signal.

Offline-first — Works without internet; uses it only as a bonus.

Pydantic — A library that checks the shape of incoming data.

SQLite — A complete database stored in one file.

Threshold — A limit above or below which we call it dangerous.

Tide gauge — A coastal device measuring sea level.

Wheel — A pre-built Python package that installs without compiling.

XSS — Cross-site scripting: tricking a page into running bad code.

References and tools

1. Allen, R. V. (1978). Automatic earthquake recognition and timing from single traces. *Bulletin of the Seismological Society of America*, 68(5). (the STA/LTA method)
2. Open-Meteo weather API — api.open-meteo.com (current weather and weather codes, no key needed).
3. OpenStreetMap and the Overpass API — overpass-api.de (shelters, hospitals, stations).
4. U.S. Geological Survey Earthquake Catalog — earthquake.usgs.gov/fdsnws/event/1 (past earthquakes).
5. FastAPI, Uvicorn and Pydantic — Python web framework, server and data validation.
6. LangGraph — library for running steps as a graph over a shared state.
7. Groq — hosted language model service used for the optional explanation sentence.
8. SQLite — sqlite.org, embedded database engine.
9. Python embeddable package — python.org, used to make the portable zip.
10. GitHub Actions — continuous-integration workflow in `.github/workflows/ci.yml`.

Project files at a glance

Table 25. Where to find each idea in the code.

IDEA	FILE
Main loop	`main.py`
All limits and switches	`config/settings.py`
Four detectors and the chain	`agents/*.py`
Alerts	`alerts/dispatcher.py`
Storage	`database/db.py`
Web server	`backend/server.py`
Screens	`frontend/index.html, mobile.css, mobile.js`
Zip builder	`build_portable.py`